When coding in thonny and making a website using drafter with multiple pages how do you make somewhere to click that will return you to the home page or a previous page?

When using the Drafter library in Python (Thonny), you can create buttons that allow users to navigate between different pages of your website.

The easiest way is to use a button that connects to another page function.

1. Create a button that returns to the home page

For example, if your home page is called `home`:

```
@route

def home():

    return Page(

        "Home Page",

        Button("Go to Home", home)

    )
```

2. Using Link() instead of a button
The main difference is that a Link looks like clickable text, while a Button looks like an actual button.

Example: Link to the home page

If your home page function is called `home`:

```
@route

def about():

    return Page(

        "About Us",

        "Welcome to our About Us page.",

        Link("Return to Home", home)

    )
```

For my website I am making I want to use a drop down selection box somewhere. How do I do this?

In Drafter, you can use a `SelectBox()` to create a dropdown selection box that lets users choose from multiple options.

For example, if you're making a construction project cost estimator, you could let users select the type of project they are working on.

1. Basic dropdown example

```
@route

def home():

    return Page(

        "Project Cost Estimator",

        SelectBox(

            "project_type",

            ["Deck", "Fence", "Garage", "Addition"]

        ),

        Button("Continue", calculate)

    )
```

How do I use the state dataclass to store data across pages

In Drafter, a state dataclass allows you to store information as a user moves between different pages of your website.

For your construction cost estimator, this is especially useful because someone might enter their project type on the home page, enter measurements on the next page, and then see their estimated cost on another page.

Instead of losing the information when they move between pages, Drafter carries the state along with them.

1. Create your state dataclass

At the top of your Python file, import `dataclass` and create a class to hold your information.

```
from dataclasses import dataclass

@dataclass

class State:

    project_type: str = ""

    material: str = ""

    project_size: int = 0
```

Each variable represents information you want to save as the user navigates your website.

2. Access the state on your pages

When using Drafter, you can include `state: State` as a parameter in your page functions.

For example:

```
@route

def home(state: State):

    return Page(

        state,

        "Construction Cost Estimator",

        "Select your project type:",

        SelectBox("project_type", ["Deck", "Fence", "Garage"]),

        Button("Continue", project_details)

    )
```

The state object is passed into the next page.

When working on this website how do I save calculations from different pages and have those values added up together on another, separate page?

For your construction cost estimator, the best way to do this is to use your State dataclass to store the individual costs from each page, then add those costs together on a separate summary page.

This allows someone to calculate different parts of a project on different pages and see one combined estimated cost at the end.

1. Add cost variables to your State dataclass

First, create variables for each category of costs you want to calculate.

For example:

```
from dataclasses import dataclass


@dataclass
class State:
    material_cost: float = 0.0
    labor_cost: float = 0.0
    equipment_cost: float = 0.0
    total_cost: float = 0.0
```

Each variable stores a different calculation.

2. Save calculations from different pages

Let's say you have three separate pages.

Materials page:

```
@route
def materials(state: State, quantity: float, price: float):
    state.material_cost = quantity * price
```

```
    return Page(
        state,
        "Materials Cost",
        "Material cost: $" + str(state.material_cost),
        Link("Next: Labor", labor)
    )
```

Labor page:

```
@route
def labor(state: State, hours: float, hourly_rate: float):
    state.labor_cost = hours * hourly_rate

    return Page(
        state,
        "Labor Cost",
        "Labor cost: $" + str(state.labor_cost),
        Link("Next: Equipment", equipment)
    )
```

Equipment page:

```
@route
def equipment(state: State, quantity: float, daily_rate: float):
    state.equipment_cost = quantity * daily_rate

    return Page(
```

```
        state,
        "Equipment Cost",
        "Equipment cost: $" + str(state.equipment_cost),
        Link("View Total", summary)
    )
```

Each page saves its calculation into the State dataclass instead of just displaying the answer.

3. Add everything together on a separate summary page

This is where the individual calculations get combined.

```
@route
def summary(state: State):
    state.total_cost = (
        state.material_cost
        + state.labor_cost
        + state.equipment_cost
    )

    return Page(
        state,
        "Project Cost Summary",
        "Materials: $" + str(state.material_cost),
        "Labor: $" + str(state.labor_cost),
        "Equipment: $" + str(state.equipment_cost),
        "----------------------",
        "TOTAL PROJECT COST: $" + str(state.total_cost),
```

```
        Link("Return Home", home)
    )
```

What calculations/equations do I need to make the concrete volume calculator make sense in the real world?

For your construction cost estimator, your concrete volume calculator should calculate how much concrete is needed to fill a specific area, then convert that amount into cubic yards because concrete is commonly ordered and priced by the cubic yard in the United States.

The most important thing is to account for the shape of the project, the dimensions, and a little extra concrete for waste.

1. Basic concrete volume equation

For a standard rectangular concrete slab, such as a driveway, patio, garage floor, or sidewalk:

$$V = L \times W \times D$$

Where:

- L = Length (feet)
- W = Width (feet)
- D = Depth (feet)
- V = Volume (cubic feet)

Important: If the user enters depth in inches, you must convert it to feet by dividing by 12.

For example, a patio that is:

- 20 feet long
- 12 feet wide
- 4 inches thick

The calculation would be:

$$20 \times 12 \times \frac{4}{12} = 80$$

The volume is 80 cubic feet.

2. Convert cubic feet to cubic yards

Concrete is generally ordered in cubic yards.

There are 27 cubic feet in one cubic yard.

$$CubicYards = \frac{CubicFeet}{27}$$

Using the patio example:

$$\frac{80}{27} = 2.96$$

You would need approximately 2.96 cubic yards before accounting for extra material.

3. Add extra concrete for waste

In real construction projects, you generally want to account for a little extra concrete because of uneven ground, spillage, and other small variations.

A reasonable estimating allowance is 5–10%, depending on the project.

For your calculator, you could use 10% as a default.

$$FinalVolume = CubicYards \times 1.10$$

What can I use to round numbers in python?

In Python, you use the `round()` function to round numbers.

1. Round to 2 decimal places

number = 3.26478

rounded_number = round(number, 2)

print(rounded_number)

2. Round to a whole number

number = 3.76

rounded_number = round(number)

print(rounded_number)

3. Always round up

If you want to always round up (useful when estimating concrete quantities), use `math.ceil()`:

```
import math

number = 3.26

rounded_number = math.ceil(number)

print(rounded_number)
```

4. Always round down

Use `math.floor()`:

```
import math

number = 3.99

rounded_number = math.floor(number)

print(rounded_number)
```

Can you help me code the summary page for my website which adds up the total costs from the wood calculator, concrete calculator, and worker calculator?

```
@route
def summary_page(state: State) -> Page:
    state.current_page = "Project Summary"
    update_total_cost(state)

    content = [
        Header("Project Cost Summary"),
        "Project: " + state.project_name,
        HorizontalRule(),
        Header("Saved Estimates")
    ]

    if len(state.estimates) == 0:
        content.append("No estimates have been saved yet.")
    else:
        for estimate in state.estimates:
            content.append(
                estimate.material_name
                + ": "
                + str(round(estimate.quantity, 2))
                + " "
                + estimate.unit
```

```
            )
            content.append(
                "Unit price: $"
                + str(round(estimate.unit_price, 2))
            )
            content.append(
                "Total cost: $"
                + str(round(estimate.total_cost, 2))
            )
            content.append(HorizontalRule())

    content.append(
        "TOTAL PROJECT COST: $"
        + str(round(state.total_cost, 2))
    )

    content.append(Header("Cost Comparison"))

    if state.total_cost > 0:
        for estimate in state.estimates:
            bar_length = int(
                estimate.total_cost / state.total_cost * 30
            )
            content.append(
                estimate.material_name
                + ": "
                + "#" * bar_length
                + " $"
                + str(round(estimate.total_cost, 2))
            )
    else:
        content.append("Save an estimate to see the comparison.")

    content.append(HorizontalRule())
    content.append(Button("Return Home", "home_page"))
    content.append(Button("Start a New Project", "clear_project"))

    return Page(state, content)
```

Can you give me unit tests using assert equal for each function in the code where possible?

Yes! You can put the unit tests at the very bottom of your existing `buildcalc.py` file, underneath the `start_server()` section.

I recommend putting them inside a function called `run_tests()`. That way, you can run the tests when you want without having them interfere with the website starting.

There is one important adjustment: your original `start_server()` currently runs immediately. We should move it to the bottom and use an `if` statement to choose whether to run tests or start the website.

1. Add the tests underneath all your website functions

```
def run_tests():
    # Test update_total_cost
    state = State("Test Project", "Home", "", [], 0.0)

    state.estimates.append(
        Estimate("Concrete", 10, "cubic yards", 100, 1000)
    )
    state.estimates.append(
        Estimate("Wood", 20, "pieces", 5, 100)
    )

    update_total_cost(state)
    assert_equal(state.total_cost, 1100.0)

    # Test add_estimate
    state = State("Test Project", "Home", "", [], 0.0)

    add_estimate(state, "Concrete", 10, "cubic yards", 100, 1000)

    assert_equal(len(state.estimates), 1)
    assert_equal(state.estimates[0].material_name, "Concrete")
    assert_equal(state.estimates[0].total_cost, 1000)

    # Test index
    state = State("Test Project", "Home", "", [], 0.0)
    result = index(state)
    assert_equal(state.current_page, "Home")
    assert_equal(type(result), Page)

    # Test home_page
    state = State("Test Project", "Home", "", [], 0.0)
    result = home_page(state)
    assert_equal(state.current_page, "Home")
    assert_equal(type(result), Page)
```

```
# Test start_project
state = State("Old Project", "Home", "", [], 100.0)
state.estimates.append(
    Estimate("Concrete", 10, "cubic yards", 10, 100)
)

result = start_project(state, "New Garage")

assert_equal(state.project_name, "New Garage")
assert_equal(len(state.estimates), 0)
assert_equal(state.total_cost, 0.0)
assert_equal(type(result), Page)

# Test concrete_page
state = State("Test Project", "Home", "", [], 0.0)
result = concrete_page(state)

assert_equal(state.current_page, "Concrete Calculator")
assert_equal(type(result), Page)

# Test save_concrete
state = State("Test Project", "Home", "", [], 0.0)
result = save_concrete(state, 27, 27, 1, 100)

assert_equal(len(state.estimates), 1)
assert_equal(type(result), Page)

# Test invalid concrete input
state = State("Test Project", "Home", "", [], 0.0)
result = save_concrete(state, -10, 10, 1, 100)

assert_equal(len(state.estimates), 0)
assert_equal(state.total_cost, 0.0)
assert_equal(type(result), Page)

# Test wood_page
state = State("Test Project", "Home", "", [], 0.0)
result = wood_page(state)

assert_equal(state.current_page, "Wood Calculator")
assert_equal(type(result), Page)

# Test save_wood
```

```
state = State("Test Project", "Home", "", [], 0.0)
result = save_wood(state, 16, 16, 8, "2x4", 5)

assert_equal(len(state.estimates), 1)
assert_equal(state.estimates[0].material_name, "Wood (2x4)")
assert_equal(type(result), Page)

# Test invalid wood input
state = State("Test Project", "Home", "", [], 0.0)
result = save_wood(state, 16, 16, 8, "2x4", -5)

assert_equal(len(state.estimates), 0)
assert_equal(state.total_cost, 0.0)
assert_equal(type(result), Page)

# Test workers_page
state = State("Test Project", "Home", "", [], 0.0)
result = workers_page(state)

assert_equal(state.current_page, "Workers Calculator")
assert_equal(type(result), Page)

# Test save_workers
state = State("Test Project", "Home", "", [], 0.0)
result = save_workers(state, 2, 8, 20)

assert_equal(len(state.estimates), 1)
assert_equal(state.estimates[0].quantity, 16)
assert_equal(state.estimates[0].total_cost, 320)
assert_equal(type(result), Page)

# Test invalid workers input
state = State("Test Project", "Home", "", [], 0.0)
result = save_workers(state, 0, 8, 20)

assert_equal(len(state.estimates), 0)
assert_equal(state.total_cost, 0.0)
assert_equal(type(result), Page)

# Test summary_page
state = State("Test Project", "Home", "", [], 0.0)

state.estimates.append(
    Estimate("Concrete", 10, "cubic yards", 100, 1000)
```

```
    )
    state.estimates.append(
        Estimate("Labor", 10, "worker-hours", 20, 200)
    )

    result = summary_page(state)

    assert_equal(state.current_page, "Project Summary")
    assert_equal(type(result), Page)

    # Test clear_project
    state = State("Old Project", "Project Summary", "", [], 500.0)
    state.estimates.append(
        Estimate("Concrete", 5, "cubic yards", 100, 500)
    )

    result = clear_project(state)

    assert_equal(state.project_name, "New Construction Project")
    assert_equal(state.current_page, "Home")
    assert_equal(len(state.estimates), 0)
    assert_equal(state.total_cost, 0.0)
    assert_equal(type(result), Page)

if __name__ == "__main__":
    run_tests()
```